Forward diffusion process. The transition $q_{t|0}(x_t|x_0)$ is a vector that represents a categorical distribution, and can be defined by a forward diffusion process $q_{t|0}(x_t|x_0) = \exp(\overline{\sigma}(t)Q)_{x_0}$, where $\overline{\sigma}(t) \in \mathbb{R}_{\geq 0}$ is the cumulative noise $\int_0^t \sigma(s)ds$ at timestep t with a value close to 0 when t is small and increasing when t growing. Lou et al. [37] use two standard transition matrices with special structures to implement matrix Q following prior work [1]:

$$Q^{\text{uniform}} = \begin{bmatrix} 1-V & 1 & \cdots & 1 \\ 1 & 1-V & \cdots & 1 \\ \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & \cdots & 1-V \end{bmatrix} \tag{34}$$

$$Q^{\text{absorb}} = \begin{bmatrix} -1 & 0 & \cdots & 0 & 0 \\ 0 & -1 & \cdots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & -1 & 0 \\ 1 & 1 & \cdots & 1 & 0 \end{bmatrix} \tag{35}$$

One can view the above diffusion process by taking small Δt Euler steps and randomly sampling the resulting transitions:

$$q_{t+\Delta t|t}(x_{t+\Delta t} = y|x_t = x) \propto \delta_{xy} + Q_t(y, x)\Delta t + O(\Delta t), \tag{36}$$

where $Q_t = \sigma(t)Q$, and $O(\Delta t)$ represents terms that tend to zero at a faster rate than Δt.

Backward generative process. To simulate the diffusion defined above, one can use the Euler strategy to derive the time reversal of the forward process:

$$q_{t-\Delta t|t}(x_{t-\Delta t} = y \mid x_t = x) \propto \delta_{xy} + R_t(y, x)\Delta t + O(\Delta t), \tag{37}$$

where R_t is the reverse transition rate matrix that can be derived using Bayes rule: $R_t(y, x) = \frac{q_t(y)}{q_t(x)} Q_t(x, y)$. Each column of R_t represents the transition probability from a token at timestep t to other tokens at timestep $t - \Delta t$. Let $p_\theta(x_{t-\Delta t} = y \mid x_t = x) = q_{t-\Delta t|t}(x_{t-\Delta t} = y \mid x_t = x)$, we have:

$$p_\theta(x_{t-\Delta t} = y \mid x_t = x) \propto \delta_{xy} + R_t^\theta(y, x)\Delta t + O(\Delta t), \tag{38}$$

where $R_t^\theta(y, x) = \sum_{x_0} q_{0|t}(x_0|x) \frac{q_{t|0}(y|x_0)}{q_{t|0}(x|x_0)} Q_t(x, y) = s_\theta(x, t)_y Q_t(x, y)$. For a sequence of random variables $\mathbf{x}$, this is inefficient because only one position is modified per step. A natural alternative has been to use *τ-leaping* [12], which performs an Euler step at each position simultaneously.

A.4 Conditional ODE solver

The sampling of continuous diffusion models can be implemented by solving the diffusion ODEs [45, 46]. Specifically, sampling by diffusion ODEs needs to discretize the following ODE [46] with t changing from T to 0:

$$\frac{\mathrm{d}\mathbf{z}_t}{\mathrm{d}t} = f(t)\mathbf{z}_t + \frac{g^2(t)}{2\sigma_t}\boldsymbol{\epsilon}_\theta(\mathbf{z}_t, t), \quad \mathbf{z}_T \sim \mathcal{N}(\mathbf{0}, \tilde{\sigma}^2\mathbf{I}). \tag{39}$$

The *data prediction model* $\mathbf{z}_\theta(\mathbf{z}_t, t)$ predicts the original data $\mathbf{z}_0$ based on the noisy $\mathbf{z}_t$, and its relationship with $\boldsymbol{\epsilon}_\theta(\mathbf{z}_t, t)$ is given by $\mathbf{z}_\theta(\mathbf{z}_t; t) := (\mathbf{z}_t - \sigma_t\boldsymbol{\epsilon}_\theta(\mathbf{z}_t, t))/\alpha_t$ [28]. Therefore, the equivalent diffusion ODE w.r.t. the data prediction model $\mathbf{z}_\theta$ is:

$$\frac{\mathrm{d}\mathbf{z}_t}{\mathrm{d}t} = \left(f(t) + \frac{g^2(t)}{2\sigma_t^2}\right)\mathbf{z}_t - \frac{\alpha_t g^2(t)}{2\sigma_t^2}\mathbf{z}_\theta(\mathbf{z}_t, t), \quad \mathbf{z}_T \sim \mathcal{N}(\mathbf{0}, \tilde{\sigma}^2\mathbf{I}), \tag{40}$$

where the coefficients $f(t) = \frac{\mathrm{d}\log\alpha_t}{\mathrm{d}t}$, $g^2(t) = \frac{\mathrm{d}\sigma_t^2}{\mathrm{d}t} - 2\frac{\mathrm{d}\log\alpha_t}{\mathrm{d}t}\sigma_t^2$ [28].

Given an initial value $\mathbf{z}_s$ at time $s > 0$ and denote $\hat{\mathbf{z}}_\theta(\hat{\mathbf{z}}_\lambda, \lambda) := \mathbf{z}_\theta(\mathbf{z}_{t_\lambda(\lambda)}, t_\lambda(\lambda))$ as the change-of-variable form of $\mathbf{z}_\theta$ for λ, the solution $\mathbf{z}_t$ at time $t \in [0, s]$ of diffusion ODEs in Eq. (40) is:

$$\mathbf{z}_t = \frac{\sigma_t}{\sigma_s}\mathbf{z}_s + \sigma_t \int_{\lambda_s}^{\lambda_t} e^\lambda \hat{\mathbf{z}}_\theta(\hat{\mathbf{z}}_\lambda, \lambda)\mathrm{d}\lambda, \tag{41}$$

which can be proved by taking derivative w.r.t. t in Eq. (41):

$$\begin{aligned}\frac{\mathrm{d}\mathbf{z}_t}{\mathrm{d}t} &= \frac{\mathrm{d}\sigma_t}{\mathrm{d}t}\frac{\mathbf{z}_s}{\sigma_s} + \frac{\mathrm{d}\sigma_t}{\mathrm{d}t}\int_{\lambda_s}^{\lambda_t} e^{\lambda}\hat{\mathbf{z}}_\theta(\hat{\mathbf{z}}_\lambda, \lambda)\mathrm{d}\lambda + \frac{\mathrm{d}\lambda_t}{\mathrm{d}t}\sigma_t e^{\lambda_t}\hat{\mathbf{z}}_\theta(\hat{\mathbf{z}}_{\lambda_t}, \lambda_t) \\ &= \frac{\mathrm{d}\sigma_t}{\mathrm{d}t}\frac{\mathbf{z}_t}{\sigma_t} + \frac{\mathrm{d}\lambda_t}{\mathrm{d}t}\sigma_t e^{\lambda_t}\hat{\mathbf{z}}_\theta(\hat{\mathbf{z}}_{\lambda_t}, \lambda_t) \\ &= \left(f(t) + \frac{g^2(t)}{2\sigma_t^2}\right)\frac{\mathbf{z}_t}{\sigma_t} - \frac{\alpha_t g^2(t)}{2\sigma_t^2}\mathbf{z}_\theta(\mathbf{z}_t, t),\end{aligned}$$

and this gives us the exact formulation as in Eq. (41).

Based on Eq. (41), the aim of an ODE solver is to approximate the exact solution at time t_i given the previous value $\mathbf{z}_{t_{i-1}}$ at time t_{i-1}. Denote $\mathbf{z}_\theta^{(n)}(\lambda) := \frac{\mathrm{d}^n \hat{\mathbf{z}}_\theta(\mathbf{z}_\lambda, \lambda)}{\mathrm{d}\lambda^n}$ as the n-th order total derivatives of $\mathbf{z}_\theta$ w.r.t. logSNR λ. Lu et al. [38, 39] show that by taking the $(k-1)$-th Taylor expansion ($k \geq 1$) at $\lambda_{t_{i-1}}$ for $\mathbf{z}_\theta$ w.r.t. $\lambda \in [\lambda_{t_{i-1}}, \lambda_{t_i}]$ and substitute it into Eq. (41) with $s = t_{i-1}$ and $t = t_i$, we have

$$\mathbf{z}_{t_i} = \frac{\sigma_{t_i}}{\sigma_{t_{i-1}}}\mathbf{z}_{t_{i-1}} + \sigma_{t_i}\sum_{n=0}^{k-1}\underbrace{\mathbf{z}_\theta^{(n)}(\hat{\mathbf{z}}_{\lambda_{t_{i-1}}}, \lambda_{t_{i-1}})}_{\text{estimated}}\underbrace{\int_{\lambda_{t_{i-1}}}^{\lambda_{t_i}} e^{\lambda}\frac{(\lambda - \lambda_{t_{i-1}})^n}{n!}\mathrm{d}\lambda}_{\text{analytically computed}} + \underbrace{\mathcal{O}(h_i^{k+1})}_{\text{omitted}}, \tag{42}$$

where the integral $\int e^{\lambda}\frac{(\lambda-\lambda_{t_{i-1}})^n}{n!}\mathrm{d}\lambda$ can be analytically computed by integral-by-parts. Therefore, to design a k-th order ODE solver, we only need to estimate the n-th order derivatives $\mathbf{z}_\theta^{(n)}(\lambda_{t_{i-1}})$ for $n \leq k-1$ after omitting the $\mathcal{O}(h_i^{k+1})$ high-order error terms. For $k = 1$, Eq. (42) becomes (after omitting the $\mathcal{O}(h_i^{k+1})$ terms)

$$\mathbf{z}_{t_i} = \frac{\sigma_{t_i}}{\sigma_{t_{i-1}}}\mathbf{z}_{t_{i-1}} + \sigma_{t_i}\mathbf{z}_\theta(\mathbf{z}_{t_{i-1}}, t_{i-1})\int_{\lambda_{t_{i-1}}}^{\lambda_{t_i}} e^{\lambda}\mathrm{d}\lambda = \frac{\sigma_{t_i}}{\sigma_{t_{i-1}}}\mathbf{z}_{t_{i-1}} - \alpha_{t_i}(e^{-h_i} - 1)\mathbf{z}_\theta(\mathbf{z}_{t_{i-1}}, t_{i-1}), \tag{43}$$

where $h_i := \lambda_{t_i} - \lambda_{t_{i-1}}$ for $i = 1, \ldots, T$.

Since DoT is conditionally trained with partial nosing, we introduce a conditional form of Eq. (43) when adapting the above ODE solver into the inference stage. For $k = 1$, this is given by:

$$\mathbf{y}_{t_i} = \frac{\sigma_{t_i}}{\sigma_{t_{i-1}}}\mathbf{y}_{t_{i-1}} - \alpha_{t_i}(e^{-h_i} - 1)\tilde{\mathbf{z}}_\theta(\mathbf{z}_{t_{i-1}}, t_{i-1}),$$

where $\mathbf{z}_{t_{i-1}} = [\mathbf{x}; \mathbf{y}_{t_i-1}]$ and $\tilde{\mathbf{z}}_\theta(\mathbf{z}_t, t)$ is used to denote the fractions of recovered $\mathbf{z}_0$ corresponding to $\mathbf{y}_0$.

B Experiment Details

B.1 Dataset Statistics

We list the statistics of our used datasets in Table 4. For the digit multiplication datasets and GSM8K dataset, we use processed datasets from Implict CoT[6] [7]. For boolean logic task, we construct the training and test dataset using the method from DyVal[7] [68]. All datasets contain 1000 test examples except GSM8K, which contains 1319 examples.

B.2 Details of Baselines

When fine-tuning GPT2, we train 40 epochs using the learning rate of `1e-4` for boolean logic and `5e-4` for others. During inference, we use greedy decoding for single decoding. For self-consistency, following the original paper [52], we apply temperature sampling with $T = 0.5$ and truncated at the top-k ($k = 40$) tokens with the highest probability for diverse generation. All GPT2-based models

[6] `https://github.com/da03/implicit_chain_of_thought`

[7] `https://github.com/microsoft/promptbench/blob/main/examples/dyval.ipynb`

Table 4: Training set size, average number of tokens in the input, intermediate, and output texts respectively when using Plaid tokenizer on the validation set and average number of rationales.

Dataset	Size	#Input token	#Intermediate token	#Output token	#Rationales
4x4	808k	16	84	15	4
5x5	808k	20	137	19	5
Boolean logic	99k	112	134	3	10
GSM8K-Aug	378k	61	34	2	2.7

use `GPT2Tokenizer` with vocabulary size of 50257. All datasets are trained using sequence length of 256 except boolean logic, which uses 384 length.

Note in Table 1, we compare Plaid DoT with the fine-tuned GPT2 small, given that the Plaid 1B [18] model exhibits similar perplexity to GPT2 small. This might put our Plaid DoT model at a disadvantage in terms of inference speed, as the parameters of Plaid 1B are nearly $10\times$ greater than those of GPT2 small.

For Transformer-scratch baseline [51], we use 6 transformer encoder layers and 6 transformer decoder layers. We employ the tokenizer from `bert-base-uncased` with a vocabulary size of 30522. The learning rate is set to `1e-5`, and we train for 60k steps with a batch size of 128.

For ChatGPT, we use OpenAI `api`[8] with the following prompt in 5-shot.

> Answer the final question following the format of the given examples.
>
> Example problems:
>
> Q: `{query}`
> A: `{answer}`
> ...
> Question to answer:
> Q:

Table 5: Prompt for ChatGPT.

Please note that the throughput of ChatGPT in Table 1 only measures the response speed of ChatGPT and does not represent the actual generation speed of the model. As a blackbox commercial product, ChatGPT may employ various optimization techniques to speedup generating responses to enhance user experiences.

B.3 DoT Implementation Details

We conduct all the experiments on NVIDIA V100-32G GPUs, and we use 8 GPUs for training and sampling. We resort to `half precision` (fp16) instead of `bfloat16` (bf16) as V100 GPU doesn't support bf16, and we don't observe any number explosion. By default, we train DoT from scratch on three datasets respectively, including the four-digit (4×4), five-digit (5×5) multiplication datasets, and the GSM8k dataset. Additionally, we fine-tune the pre-trained model Plaid-1B on the GSM8K dataset with DoT to explore its effectiveness further.

For DoT trained from scratch. We use 12 layers of transformer and `bert-base-uncased` vocabulary. We preprocess the four-digit (4×4) and five-digit (5×5) multiplication datasets to prepare for the training process of the DoT multi-path variant, and sampling from it. The learning rate is `1e-4` and we train for 60k steps with the batch size of 128 and max sequence length of 128. For digit

[8] `https://platform.openai.com/docs/api-reference`